



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

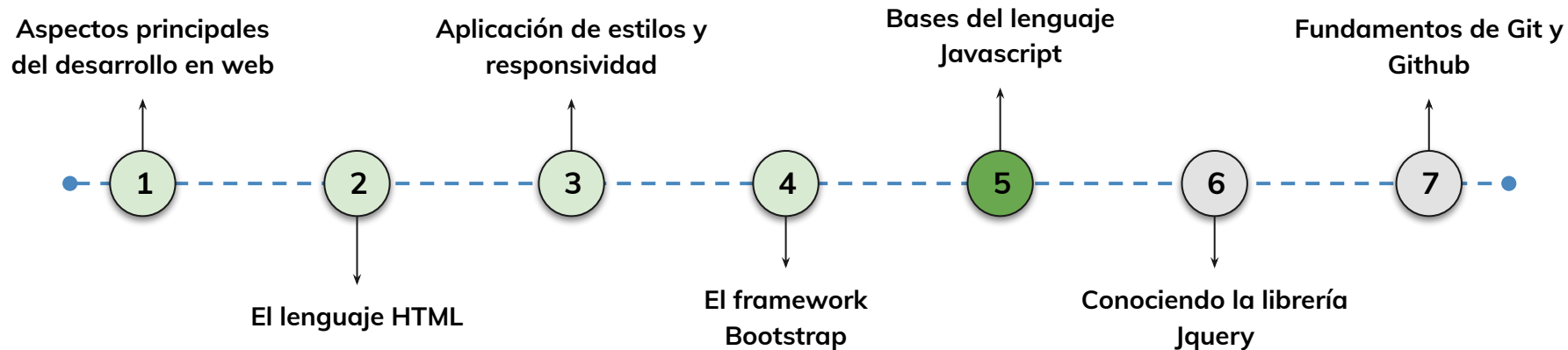


› Bases del lenguaje Javascript - Parte I

AE 2.5: Codificar en lenguaje javascript rutinas simples para la personalización de eventos sencillos dentro de un documento html dando solución al problema planteado

Hoja de ruta

¿Cuáles skills conforman el programa?



Learning Path

¿Cuáles temas trabajaremos hoy?

12.

Bases de JavaScript

JavaScript es un lenguaje de programación fundamental para el desarrollo web, permitiendo agregar interactividad y dinamismo a las aplicaciones.

Introducción a JavaScript

Historia y relevancia de JavaScript

Variables en JavaScript

Operaciones y estructuras de control

Expresiones aritméticas y operadores

Condicionales en JavaScript

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender la historia y el origen de JavaScript como un lenguaje de programación esencial en el desarrollo web.
- Reconocer la importancia de JavaScript en el contexto del desarrollo web y su papel en la creación de experiencias interactivas para los usuarios.
- Aprender el concepto de variables en JavaScript y cómo se utilizan para almacenar y manipular datos en las aplicaciones web
- Aprender cómo trabajar con expresiones aritméticas en JavaScript.
- Aprenderemos sobre las estructuras if, else if y else y cómo se utilizan.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Incorporamos el componente form de bootstrap de manera más rápida para crear formularios
- Aprendimos a modificar y agregar menú a partir del componente navbar

Manejo de JavaScript

< > Breve historia de JavaScript

JavaScript es un lenguaje de programación que fue creado en 1995 por Brendan Eich, quien trabajaba en Netscape Communications Corporation en ese momento. Inicialmente, el lenguaje se llamaba "LiveScript" y fue desarrollado como una forma de agregar interactividad y funcionalidad a las páginas web.



< > Algoritmos

- En programación, un algoritmo es un conjunto de procedimientos o funciones ordenados que se necesitan para realizar cierta operación o acción. Por ejemplo, en una suma el algoritmo implica tomar un dato, sumarlo a otro y obtener un resultado.
- Pensar en algoritmos es una práctica que debemos fortalecer como desarrolladores. Consiste en encarar un problema complejo y dividir su resolución en diversos pasos, pensar cómo resolver cada uno y luego secuenciarlos correctamente para llegar al resultado esperado.



< > Relevancia de Javascript

JavaScript tiene sus propias reglas para la sintaxis, aunque respeta los estándares de muchos lenguajes de programación lógicos.

Nuestro código JavaScript se asocia al archivo HTML que se carga en el navegador. Tenemos dos maneras de escribir código JavaScript en nuestras aplicaciones web.

JavaScript es un lenguaje de programación de alto nivel que desempeña un papel fundamental en el desarrollo web y en la creación de aplicaciones interactivas en el navegador.



< > Relevancia de Javascript

Interactividad en el navegador

JavaScript permite agregar interactividad y dinamismo a las páginas web. Gracias a este lenguaje, es posible crear efectos visuales, animaciones, juegos, validaciones de formularios y muchas otras características que mejoran la experiencia del usuario en el navegador.

Desarrollo web front-end:

JavaScript es el principal lenguaje utilizado en el desarrollo de la capa de presentación o front-end de las aplicaciones web. Junto con HTML y CSS, JavaScript permite construir interfaces de usuario interactivas y receptivas, proporcionando una experiencia de usuario fluida e intuitiva.



< > Relevancia de Javascript

Frameworks y bibliotecas populares:

Existen numerosos frameworks y bibliotecas de JavaScript, como React, Angular y Vue.js, que facilitan y agilizan el desarrollo de aplicaciones web complejas. Estas herramientas proporcionan una arquitectura modular, componentes reutilizables y una forma eficiente de manejar la lógica del front-end.

Desarrollo web back-end:

JavaScript no se limita solo al front-end, ya que también puede utilizarse para el desarrollo del lado del servidor o back-end. Node.js, una plataforma basada en JavaScript, permite ejecutar código JavaScript en el servidor, lo que brinda la posibilidad de crear aplicaciones web completas utilizando un solo lenguaje en todos los niveles.



< > Relevancia de Javascript

Compatibilidad con múltiples navegadores:

JavaScript es compatible con la mayoría de los navegadores web modernos, lo que garantiza que las aplicaciones desarrolladas en JavaScript sean accesibles y se ejecuten correctamente en diferentes entornos.

Integración con tecnologías emergentes:

JavaScript se ha adaptado y evolucionado para integrarse con tecnologías emergentes como la inteligencia artificial, la realidad virtual, la realidad aumentada y el Internet de las cosas. Esto permite desarrollar aplicaciones avanzadas y aprovechar las últimas tendencias tecnológicas.



LIVE CODING

Ejemplo en vivo

¿En qué consistirá la Demo?

Reflexionemos sobre el papel de JavaScript en la web y en la programación en general, analizando su importancia, evolución y desafíos futuros.

Preguntas para debatir:

1. Si tuvieras que enseñar a alguien cómo pensar en algoritmos, ¿qué estrategia utilizarías?
2. ¿Por qué crees que es importante dividir un problema en pasos antes de escribir código? ¿Has experimentado alguna vez las consecuencias de no hacerlo?
3. ¿Qué desafíos crees que enfrenta JavaScript en el desarrollo del front-end moderno?
4. ¿Cómo podrías mejorar la experiencia del usuario utilizando JavaScript de manera creativa?

Tiempo: 10 Minutos

< > ¿Cómo escribir código JS?

Dentro de un archivo html, podemos escribir código de JavaScript entre medio de las etiquetas <script>

Al agregar comentarios en tu código JavaScript puedes hacerlo más legible y comprensible. Los comentarios son líneas de código que no se ejecutan y se utilizan para agregar notas o explicaciones.

En JavaScript, puedes usar // para comentarios de una línea y /* ... */ para comentarios de varias líneas.

```
<script  
src="ruta/al/archivo/script.js">  
</script>
```



< > ¿Cómo escribir código JS?

Ejemplo

```
<script>
    //aca va el código en JS
    // Esto es un comentario de una línea

    /*
        Esto es un comentario
        de varias líneas
    */
</script>
```



< > Variables

Puntos a tener en cuenta:

- Asegúrate de seguir la sintaxis correcta del lenguaje JavaScript. Presta atención a los paréntesis, llaves, puntos y comas. Un error de sintaxis puede provocar que el código no funcione correctamente.
- En JavaScript, las variables se utilizan para almacenar y manipular datos. Para declarar una variable en JavaScript, se utilizan las palabras clave `var`, `let` o `const`, seguidas del nombre de la variable.



< > Variables

- **var:** Era la forma tradicional de declarar variables en JavaScript, pero se ha vuelto menos común con la introducción de let y const.

```
var edad = 25;
```

- **let:** En ES6, let permite declarar variables con alcance de bloque, es decir, solo existen dentro del bloque donde se definen, como en funciones, bucles o condicionales.

```
let nombre = 'Juan';  
const nacionalidad = 'Argentina';
```

- **const:** También introducido en ES6, const se utiliza para declarar constantes, es decir, variables que no pueden cambiar su valor una vez asignado. Al igual que let, const tiene un alcance de bloque.

```
const pi = 3.1416;
```



< > Variables

Es importante tener en cuenta que `let` y `const` siguen las mejores prácticas actuales para declarar variables en JavaScript. Se recomienda utilizar `const` siempre que sea posible, ya que ayuda a prevenir la reasignación accidental de valores. Si sabes que el valor de una variable cambiará, puedes utilizar `let`.

Además de declarar variables, puedes asignarles valores utilizando el operador de asignación `=`:

```
let mensaje = 'Hola, mundo!';
```

Una vez que una variable ha sido declarada, puedes utilizarla en tu código, ya sea para realizar operaciones, imprimir su valor en la consola o utilizarla en otras expresiones.

```
let x = 10;  
let y = 5;  
let suma = x + y;  
console.log(suma);  
// Imprime 15 en la consola
```



< > Variables

Declaración

En JavaScript, las variables se declaran con `var`, `let` o `const`, seguidas de un nombre. No deben contener espacios; se recomienda usar `_` o `camelCase`.

```
var miVariable;  
let nombreCompleto;  
const numeroDeTelefono;
```

Asignación

La asignación de valores a una variable, se realiza utilizando el operador de asignación, que es el signo igual (`=`). Este operador asigna el valor a la variable en el lado izquierdo de la expresión.

```
var nombre = "Juan"; // Asigna el valor  
"Juan" a la variable nombre  
let edad = 25;  
// Asigna el valor 25 a la variable edad  
const pi = 3.1416;  
// Asigna el valor 3.1416 a la constante pi
```



< > Variables

Inicializar variables

En JavaScript es posible declarar una variable y asignarle un valor inicial en el mismo proceso. Esto se conoce como inicialización de variable..

La inicialización de variables en el mismo proceso puede ser útil cuando conoces el valor inicial que deseas asignar a la variable al momento de declararla. Esto evita la necesidad de realizar una asignación por separado después de la declaración.

```
var edad = 25;  
// Declaración y asignación inicial de  
la variable edad  
let nombre = "Juan";  
// Declaración y asignación inicial de  
la variable nombre  
const pi = 3.1416;  
// Declaración y asignación inicial de  
la constante pi
```



< > Funciones Nativas

Las funciones nativas, también conocidas como funciones incorporadas o funciones predefinidas, son funciones que vienen incluidas en JavaScript y están disponibles para su uso directo sin necesidad de definirlas previamente

Estas funciones proporcionan una amplia gama de funcionalidades y permiten realizar diversas tareas comunes de manera eficiente.



< > ¿Cómo escribir código JS?

console.log():

Esta función se utiliza para imprimir mensajes en la consola del navegador o en la consola del entorno de desarrollo. Es útil para realizar pruebas y depurar el código, ya que puedes imprimir valores y mensajes para verificar el flujo de ejecución.

```
console.log("Mensaje de prueba");
```

alert():

La función alert() muestra una ventana emergente en el navegador con un mensaje para el usuario. Es útil para mostrar mensajes de advertencia, notificaciones o solicitar confirmación del usuario.

```
alert(";Hola, mundo!");
```



< > ¿Cómo escribir código JS?

confirm():

La función `confirm()` muestra una ventana emergente en el navegador con un mensaje y botones de confirmación ("Aceptar" y "Cancelar"). Es útil para obtener la confirmación del usuario en situaciones específicas.

```
confirm("¿Está seguro de eliminar este elemento?");
```

Prompt:

El `prompt` en JavaScript es una función que muestra un cuadro de diálogo al usuario para solicitar una entrada de datos. Proporciona una forma sencilla de obtener información del usuario a través de una ventana emergente en el navegador.

```
prompt("Por favor, ingrese su nombre:")
```



< > ¿Cómo escribir código JS?

parseInt() y parseFloat():

Estas funciones se utilizan para convertir una cadena de texto en un número entero o de punto flotante, respectivamente. Son útiles cuando se necesita manipular datos numéricos ingresados como cadenas de texto.

```
parseInt("10");  
parseFloat("3.14");
```

Concatenación:

La concatenación en JavaScript es la unión de cadenas de texto en una sola. Se puede realizar con el operador +, comúnmente usado para formar mensajes y etiquetas.

```
let nombre = "Juan";  
let apellido = "Pérez";  
let nombreCompleto = nombre + " " +  
    apellido;  
console.log(nombreCompleto);  
// Resultado: "Juan Pérez"
```



< > ¿Cómo escribir código JS?

Utilizando plantillas de cadena (template literals):

Las plantillas de cadena son una forma más moderna y conveniente de realizar concatenación en JavaScript. Se utilizan utilizando comillas graves (``) y permiten incrustar variables o expresiones dentro del texto utilizando \${}.

La concatenación de cadenas es una operación fundamental en JavaScript y se utiliza ampliamente en el desarrollo de aplicaciones web para generar y manipular texto dinámicamente.

```
let nombre = "Juan";  
let mensaje = `Hola, ${nombre}!`;   
console.log(mensaje);  
// Resultado: "Hola, Juan!"
```



LIVE CODING

Ejemplo en vivo

¿En qué consistirá la Demo?

A continuación vamos repasar cómo declarar variables y asignarles valores en tiempo real.

Para ello vamos a crear un algoritmo que solicite al usuario su nombre y edad, y luego muestre un mensaje de bienvenida en la consola.

Tiempo: 10 Minutos



Momento:

Time-out!



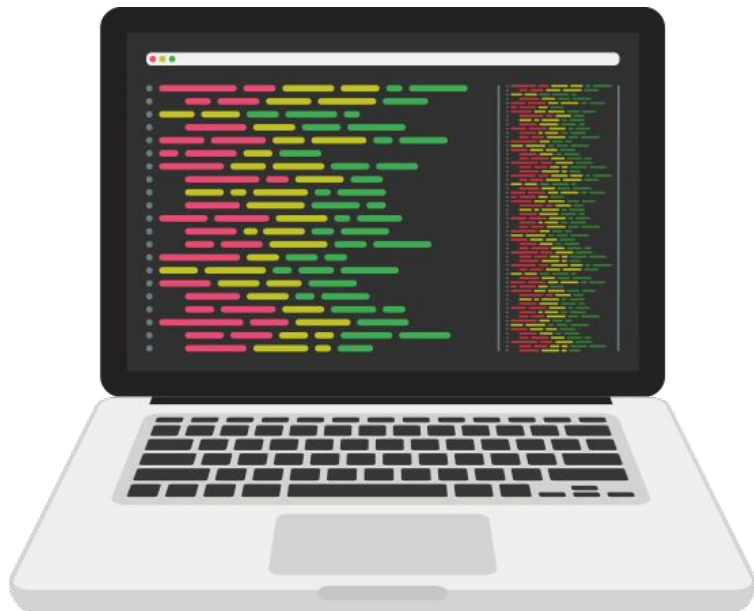
5 -10 min.



< > Expresiones aritméticas

En JavaScript, las expresiones aritméticas se utilizan para realizar operaciones matemáticas y calcular resultados numéricos.

Las expresiones aritméticas pueden involucrar operadores matemáticos como suma (+), resta (-), multiplicación (*), división (/), resto (%) y exponente (**).



< > Estructura Básica en HTML

Todo documento web en HTML sigue una estructura básica compuesta por las etiquetas principales:

<!DOCTYPE html>, <html>, <head> y <body>.

Dentro de esta estructura podemos incluir un bloque <script> que permite agregar código JavaScript para ejecutar acciones, mostrar mensajes o realizar cálculos lógicos.

De esta forma, HTML define el contenido, mientras que JavaScript agrega la interactividad y la lógica al sitio.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Mi primer documento HTML + JS</title>
</head>
<body>
  <h1>Ejemplo de integración HTML y JavaScript</h1>
  <p>Este es un documento HTML con espacio para
incluir código JavaScript.</p>

  <!-- Aquí va la lógica del script -->
  <script>
    // Aquí irá el código JavaScript
  </script>
</body>
</html>
```



LIVE CODING

Ejemplo en vivo

¿En qué consistirá la Demo?

Crearemos un pequeño script en HTML que pida el nombre del usuario y muestre un mensaje de bienvenida al ingresar a la página.

Paso 1: Crear la estructura base: Armar el documento HTML con sus etiquetas principales.

Paso 2: Agregar el bloque de script: Escribir el código dentro de `<script>` al final del body.

Paso 3: Probar en el navegador: Ingresar el nombre y ver el mensaje de bienvenida.

Tiempo: 10 Minutos

< > Expresiones aritméticas

Los operadores aritméticos básicos, JavaScript también proporciona operadores de asignación compuestos que permiten combinar una operación aritmética con una asignación en una sola expresión.

Se utilizan diferentes operadores aritméticos para realizar operaciones matemáticas básicas. Es importante tener en cuenta que las operaciones se realizan de acuerdo con las reglas de precedencia matemática estándar.

```
let x = 5;
let y = 3;
let suma = x + y; // Suma: 8
let resta = x - y; // Resta: 2
let multiplicacion = x * y;
// Multiplicación: 15
let division = x / y;
// División: 1.6666666666666667
let resto = x % y; // Resto: 2
let exponente = x ** y; // Exponente: 125
```



< > Expresiones aritméticas

Suma (+)

La operación de suma se realiza utilizando el signo (+) para sumar los valores numéricos que queremos combinar.

```
let numero1 = 10;
let numero2 = 50;
console.log(10 + 50);
console.log(numero1 + numero2)

let resultadoSuma = numero1 + numero2;
console.log(resultadoSuma)
```

Resta (-)

De manera análoga, la operación de resta se realiza utilizando el signo (-) para restar los valores numéricos que deseamos operar.

```
let numero1 = 100;
let numero2 = 50;
console.log(100 - 50);
console.log(numero1 - numero2)

let resultadoResta = numero1 - numero2;
console.log(resultadoResta)
```



< > Expresiones aritméticas

Multiplicación (*)

La operación de multiplicación se realiza utilizando el signo de multiplicación, el asterisco (*), para multiplicar los valores numéricos que deseamos operar.

```
let numero1 = 70;
let numero2 = 70;
console.log(70 * 7);
console.log(numero1 * numero2)

let resultadoMultiplicacion = numero1 *
numero2;
console.log(resultadoMultiplicacion)
```

División (/)

La operación división se realiza ubicando signo de la división, la barra inclinada hacia la derecha (/), entre los números que queremos dividir.

```
let numero1 = 80;
let numero2 = 4;
console.log(80 / 4);
console.log(numero1 / numero2)

let resultadoDivision = numero1 /
numero2;
console.log(resultadoDivision)
```



< > Expresiones aritméticas

% (módulo):

Calcula el resto de la división entre x e y. En el ejemplo, `x % y` da como resultado 2, que es el resto de la división de 5 entre 3.

** (exponente):

Eleva x a la potencia y. En el ejemplo, `x ** y` da como resultado 125, que es 5 elevado a la potencia 3.

```
let resto = x % y;  
// Resto: 2
```

```
let exponente = x ** y;  
// Exponente: 125
```



< > Operadores de incremento y decremento

En JavaScript, existen operadores especiales que permiten aumentar (++) o disminuir (--) el valor de una variable de forma abreviada, en lugar de escribir una suma o resta manualmente.

Estos operadores son muy útiles en bucles y en situaciones donde es necesario modificar el valor de una variable de manera sencilla.

```
let x= 10;

// Forma tradicional:
x = x + 1; // Ahora x = 11
x = x - 1; // Ahora x = 10

// Forma abreviada:
x++; // Ahora x = 11
x--; // Ahora x = 10
```



< > Operadores de Asignación

Los operadores de asignación en JavaScript son utilizados para asignar valores a variables. Estos operadores permiten no solo asignar valores simples, sino también realizar operaciones en variables mientras se les asigna un nuevo valor.

Se utilizan operadores de asignación compuestos junto con los operadores aritméticos básicos para actualizar el valor de la variable realizando la operación aritmética correspondiente y asignando el resultado a la misma variable.

```
let z = 10;  
z += 5; // Equivale a z = z + 5; (z = 15)  
z -= 3; // Equivale a z = z - 3; (z = 12)  
z *= 2; // Equivale a z = z * 2; (z = 24)  
z /= 4; // Equivale a z = z / 4; (z = 6)  
z %= 2; // Equivale a z = z % 2; (z = 0)
```



< > Operadores de comparación

Igualdad (==):

Compara si dos valores son iguales, sin tener en cuenta el tipo de dato.

Devuelve true si son iguales y false si no lo son.

Ambos valores son considerados iguales, a pesar de que uno es un número y el otro es una cadena de texto.

```
let x = 5;  
let y = "5";  
  
console.log(x == y); // Devuelve true
```

Igualdad estricta (===):

Compara si dos valores son iguales, teniendo en cuenta tanto el valor como el tipo de dato.

Devuelve true si son iguales en valor y tipo, y false si no lo son.

La comparación `x === y` devuelve false porque los valores son iguales, pero los tipos de dato son diferentes.

```
let x = 5;  
let y = "5";  
  
console.log(x === y); // Devuelve false
```

< > Operadores de comparación

Desigualdad (!= o !==):

Compara si dos valores no son iguales. Devuelve true si son diferentes y false si son iguales.

El operador !== también verifica el tipo de dato.

Tanto la comparación `x != y` como `x !== y` devuelven true porque los valores son diferentes.

```
let x = 5;
let y = 10;

console.log(x != y); // Devuelve true
console.log(x !== y); // Devuelve true
```

Otros operadores de comparación:

Además de la igualdad y desigualdad, existen otros operadores de comparación como `>`, `<`, `>=` y `<=`, que comparan si un valor es mayor, menor, mayor o igual, o menor o igual, respectivamente.

```
let x = 5;
let y = 10;

console.log(x > y); // Devuelve false
console.log(x < y); // Devuelve true
console.log(x >= y); // Devuelve false
console.log(x <= y); // Devuelve true
```

< > Operadores Lógicos

Operador AND (&&):

El operador AND devuelve true si ambos operandos son true, y devuelve false en cualquier otro caso.

El operador **&&** se utiliza para evaluar dos expresiones lógicas y devuelve true si ambas expresiones son verdaderas.

```
let x = 5;
let y = 10;

console.log(x > 0 && y > 0);
// Devuelve true
console.log(x > 0 && y < 0);
// Devuelve false
```

Operador OR (||):

El operador OR devuelve true si al menos uno de los operandos es true, y devuelve false solo si ambos operandos son false.

El operador **||** se utiliza para evaluar dos expresiones lógicas y devuelve true si al menos una de las expresiones es verdadera.

```
let x = 5;
let y = 10;

console.log(x > 0 || y > 0);
// Devuelve true
console.log(x < 0 || y < 0);
// Devuelve false
```


< > Operadores Lógicos

Operador OR (||):

El operador OR devuelve true si al menos uno de los operandos es true, y devuelve false solo si ambos operandos son false.

El operador || se utiliza para evaluar dos expresiones lógicas y devuelve true si al menos una de las expresiones es verdadera.

```
let x = 5;
let y = 10;

console.log(x > 0 || y > 0);
// Devuelve true
console.log(x < 0 || y < 0);
// Devuelve false
```



< > Operadores Lógicos

Operador NOT (!):

El operador NOT invierte el valor de una expresión booleana. Si la expresión es true, el operador NOT devuelve false, y si la expresión es false, el operador NOT devuelve true.

El operador ! se utiliza para negar una expresión lógica. Convierte true en false y viceversa.

```
let x = 5;
let y = 10;

console.log(!(x > 0));
// Devuelve false
console.log(!(y < 0));
// Devuelve true
```



< > Sentencias condicionales

Sentencia if-else:

La sentencia if-else es una estructura de control en programación que permite tomar decisiones basadas en una condición. A diferencia de la sentencia if, que ejecuta un bloque de código si una condición es verdadera y luego continúa con el resto del programa, la sentencia if-else permite ejecutar un bloque de código si la condición es verdadera y otro bloque de código si la condición es falsa.



< > Sentencias condicionales

```
if (condición) {  
  // Este bloque de código se ejecutará si la condición es verdadera.  
} else {  
  // Este bloque de código se ejecutará si la condición es falsa.  
}  
  
let hora = 13;  
  
if (hora < 12) {  
  console.log("Buenos días");  
} else {  
  console.log("Buenas tardes");  
}
```



< > Sentencias condicionales

Operador AND (&&) con IF/ELSE:

El operador && devuelve true si ambas condiciones son verdaderas. Si alguna de las condiciones es falsa, devuelve false.

La condición dentro del if utiliza el operador && para verificar si la edad es mayor o igual a 18 y si tiene Licencia es true. Si ambas condiciones son verdaderas, se muestra el mensaje "puedes conducir legalmente". Si alguna de las condiciones es falsa, se muestra el mensaje "No cumples con los requisitos para conducir".

```
let edad = 25;
let tieneLicencia = true;

if (edad >= 18 && tieneLicencia) {
    console.log("puedes conducir legalmente.");
} else {
    console.log("No cumples con los requisitos para conducir.");
}
```



< > Sentencias condicionales

Operador OR (||) con IF/ELSE:

El operador || devuelve true si al menos una de las condiciones es verdadera. Solo devuelve false si ambas condiciones son falsas.

La condición dentro del if utiliza el operador || para verificar si esEstudiante es true o si esEmpleado es true. Si al menos una de las condiciones es verdadera, se muestra el mensaje "Tenes acceso a descuentos especiales". Si ambas condiciones son falsas, se muestra el mensaje "No tenes acceso a descuentos especiales".

```
let esEstudiante = false;
let esEmpleado = true;

if (esEstudiante || esEmpleado) {
    console.log("Tenes acceso a descuentos especiales.");
} else {
    console.log("No tenes acceso a descuentos especiales.");
}
```



< > Switch

expresion es la expresión que se va a evaluar.

case valor1: es un caso específico que se compara con la expresión. Si la expresión coincide con valor1, se ejecuta el código dentro de ese caso.

break; se utiliza para salir del switch después de ejecutar el código de un caso.

default: es un caso opcional que se ejecuta si ninguno de los casos coincide con la expresión.



< > Switch

```
switch (expresion) {  
  case valor1:  
    //Bloque de código a ejecutar si la expresion es igual a valor1  
    break;  
  case valor2:  
    //Bloque de código a ejecutar si la expresion es igual a valor2  
    break;  
  case valor3:  
    //Bloque de código a ejecutar si la expresion es igual a valor3  
    break;  
  // ...  
  default:  
    //Bloque de código a ejecutar si ninguno de los casos se cumple  
}
```



< > Sentencias condicionales

El operador ternario consta de tres partes:

La condición es una expresión que se evalúa como verdadera (true) o falsa (false).

- La expresión1 se ejecuta si la condición es verdadera.
- La expresión2 se ejecuta si la condición es falsa.

```
condicion ? expresion1 : expresion2;
```



< > Sentencias condicionales

El operador ternario consta de tres partes:

La condición es una expresión que se evalúa como verdadera (true) o falsa (false).

- La expresión1 se ejecuta si la condición es verdadera.
- La expresión2 se ejecuta si la condición es falsa.

```
condicion ? expresion1 : expresion2;
```

```
let num1 = 7;
let num2 = 5;

//Operador ternario: (condición ?
verdadero : falso)

let rta = num1 > num2 ? "el 1" : "el 2";

console.log(rta);
```



LIVE CODING

Ejemplo en vivo

¿En qué consistirá la Demo?

Los alumnos comprenderán cómo aplicar operadores matemáticos y lógicos en combinación con estructuras condicionales para tomar decisiones en tiempo real dentro de un programa en JavaScript.

Paso 1: Deberán declarar dos números ingresados por el usuario y aplicarán operaciones matemáticas como suma, resta, multiplicación, división y módulo.

Paso 2: Utilizarán operadores de asignación para actualizar valores de variables y mostrar cómo funcionan en la práctica.

Tiempo: 10 Minutos

LIVE CODING

Ejemplo en vivo

Paso 3: El programa evaluará si los números cumplen ciertas condiciones, como si son positivos, negativos, pares o múltiplos entre sí.

Paso 4: Los estudiantes escribirán una estructura condicional que determine si un número es mayor, menor o igual al otro e imprimirán el resultado en la consola.

Paso 5: Se implementará un menú donde el usuario podrá seleccionar una operación matemática y el programa ejecutará la acción correspondiente.

Tiempo: 10 Minutos



Ejercicio N° 1

Estructura de control

Estructura de control

Contexto: 🙌

Eres un desarrollador de software y te han asignado la tarea de crear un sistema de inicio de sesión simple utilizando JavaScript y la estructura de control if.

Consigna: ✍️

Crear un sistema de inicio de sesión que solicita al usuario un nombre de usuario y una contraseña, verifica si son correctos y muestra un mensaje de bienvenida o error en función de la verificación.

Tiempo 🕒: 15 Minutos

Estructura de control

Paso a paso: ⚙️

1. Crea un nuevo archivo JavaScript (puedes nombrarlo como "login.js").
2. Declara dos variables: `usuarioValido` y `contrasenaValida`. Asigna un nombre de usuario y una contraseña válidos como cadenas de texto.
3. Utiliza la función `prompt` para solicitar al usuario que ingrese su nombre de usuario y contraseña. Almacena estos valores en variables.
4. Utiliza la estructura de control `if` para verificar si el `usuarioIngresado` y la `contrasenaIngresada` coinciden con las credenciales válidas (`usuarioValido` y `contrasenaValida`).
5. Utiliza `console.log` para mostrar un mensaje de bienvenida si las credenciales son correctas o un mensaje de error si son incorrectas.

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprender la importancia de JavaScript en el desarrollo web moderno.**
- ✓ **Explorar el concepto de variables en JavaScript y cómo se utilizan para almacenar y manipular datos.**
- ✓ **Comprender y aplicar expresiones aritméticas, operadores de asignación y estructuras condicionales (if, if-else) en JavaScript.**
- ✓ **Utilizar la sentencia switch para realizar múltiples comprobaciones de igualdad de manera eficiente.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

